



Figure 7: Logout message

```

else if(mysql_result($res,0,'duration') > $time_out){
    print("<center><h1>Session timed out!</h1></center><br />");
    $res = mysql_query("select @m:=max(session_id) from session_log where remote_ip='$remote_ip';", $con);
    $res = mysql_query("update session_log set status=2 where remote_ip='$remote_ip' and session_id=@m;", $con) or die(mysql_error());
    mysql_close($con);
    print("<a href='\"dbloginform.html\"'>Go to Login Page</a>");
    exit(0);
}

$id = mysql_result($res,0,'user_id');
$res = mysql_query("select name from user where id=$id;", $con) or die(mysql_error());
$user = mysql_result($res,0,'name');
$res = mysql_query("update session_log set status=2, last_access=now() where remote_ip='$remote_ip' and session_id=$session_id;", $con) or die(mysql_error());
print("<center><h1>Successfully logged out $user!</h1><br />");
print("<a href='\"dbloginform.html\"'>Return to login page</a></center>");

$res = mysql_query("update session_log set status=2 where (now()-last_access)/60 >= $time_out;", $con) or die(mysql_error());
mysql_close($con);
logoutUser();
?>

```

The logout script sets the status of the session to *Expired* and the last-accessed time to the current time. If the session was already closed or timed out, the logout script will display the appropriate message and terminate. Figure 9 shows the entries in the *session_log* table. This table would serve both as a record of the



Figure 8: Status message after logging out

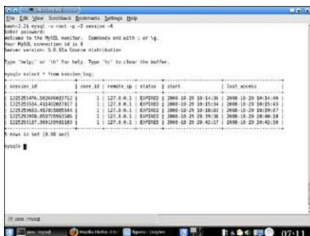


Figure 9: Session log maintained by the server

login/last-accessed times for users as well as a safe mechanism to hold current session details.

Pros and cons of server-side sessions

Forming sessions from the server side provides the safest strategy for the formation of sessions. The formation of sessions is fail-safe, since it is completely independent of cookies. Normally, server machines are better protected from intrusions. Hence, the chance of unauthorised people getting hold of session details is very limited. However, it takes a bit more coding effort on the part of the programmer.

After getting through the client- and server-side session management strategies, I'm sure you now will be able to easily protect Web content from the eyes of unauthorised users in your projects. Although server-side sessions are a bit tedious and time consuming, the results are more rewarding than the cookie-based implementation. **END** 

By: V. Nagaradjane

The author is a freelance programmer and can be contacted at nagaradjanev@rediffmail.com